

# Teach Module 20: Logging Frameworks and Diagnostics

## Module Overview

Module 20 focuses on logging and diagnostics in Java services.

Logging is how an application explains what happened while it was running. In production, developers usually cannot attach a debugger or pause the application. Logs become the evidence trail for understanding requests, failures, performance issues, security events, and integration problems.

Good logs are intentional, searchable, consistent, and safe.

## Learning Goals

By the end of this module, you should be able to:

- Explain why logging matters in enterprise Java applications.
- Use logging levels correctly.
- Use SLF4J and Logback in Java or Spring Boot applications.
- Write parameterized and structured logs.
- Use correlation IDs to trace requests.
- Log exceptions correctly with stack traces.
- Avoid logging sensitive data.
- Diagnose runtime issues from logs.

## 1. Why Logging Matters

Logs help answer questions such as:

- Did the application start correctly?
- Which request caused the issue?
- Which user, order, payment, or transaction was involved?
- Did validation fail?
- Did the database call fail?
- Did an external API fail?
- Was this one isolated request or a wider system problem?

Poor logging creates confusion. Good logging makes diagnosis faster.

Avoid using `System.out.println()` for application diagnostics. Use a logging framework instead.

## 2. Logging Levels

Most Java logging frameworks use these levels:

```
TRACE - extremely detailed internal flow
DEBUG - useful for developers during troubleshooting
INFO  - important normal application events
WARN  - unexpected but recoverable situations
ERROR - failures that need attention
```

Example:

```
log.info("Order created successfully: orderId={}", orderId);

log.warn("Payment retry required: orderId={}, attempt={}", orderId, attempt);

log.error("Payment processing failed: orderId={}", orderId, exception);
```

Use the level that matches the situation. Do not log everything as `INFO` or everything as `ERROR` .

### 3. SLF4J and Logback

In many Java and Spring Boot applications:

- SLF4J is the logging API.
- Logback is the logging implementation.

SLF4J is what your code calls. Logback decides how logs are formatted, filtered, and written.

Basic Java example:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class PaymentService {
    private static final Logger log = LoggerFactory.getLogger(PaymentService.class);

    public void processPayment(String orderId) {
        log.info("Processing payment for orderId={}", orderId);
    }
}
```

Spring Boot with Lombok:

```
@Slf4j
@Service
public class PaymentService {
    public void processPayment(String orderId) {
        log.info("Processing payment for orderId={}", orderId);
    }
}
```

### 4. Parameterized Logging

Prefer parameterized logging:

```
log.info("Customer lookup completed: customerId={}", customerId);
```

Avoid string concatenation:

```
log.info("Customer lookup completed: customerId=" + customerId);
```

Parameterized logging is cleaner and avoids unnecessary string construction when a log level is disabled.

## 5. Structured Logging

Traditional log:

```
Payment failed for order 123
```

Structured log:

```
event=payment_failed orderId=123 reason=card_declined status=FAILED
```

Structured logs are easier to search and filter in tools such as Splunk, ELK, Datadog, or CloudWatch.

Example:

```
log.warn("Payment declined: orderId={}, reason={}, status={}",
        orderId,
        declineReason,
        paymentStatus);
```

The goal is to include meaningful fields, not just a sentence.

## 6. Correlation IDs

A correlation ID is a unique identifier attached to one request as it moves through the system.

Example log sequence:

```
correlationId=abc-123 request=POST /orders
correlationId=abc-123 validating order
correlationId=abc-123 calling payment service
correlationId=abc-123 payment approved
```

Correlation IDs are especially useful when one user action touches multiple services, such as:

- frontend
- API gateway
- Spring Boot service
- database
- Kafka
- another microservice

Without a correlation ID, debugging distributed systems becomes much harder.

## 7. What To Log

Useful events to log include:

- Application startup and shutdown.
- Important incoming requests.
- Business events such as order created, payment failed, or account locked.
- External API calls and failures.
- Validation failures.
- Retry attempts.
- Security-relevant events.
- Unexpected exceptions.
- Timing or performance issues.

Do not log every tiny method call in production unless you are intentionally using `DEBUG` or `TRACE` .

## 8. What Not To Log

Never log sensitive data such as:

- passwords
- access tokens
- refresh tokens
- full credit card numbers
- Social Security numbers
- private keys
- authentication cookies
- confidential personal data unless explicitly approved

Bad example:

```
log.info("Login request: username={}, password={}", username, password);
```

Better:

```
log.info("Login attempt received: username={}", username);
```

For payment flows, do not log raw payment tokens.

## 9. Exception Logging

A common mistake is logging only the exception message:

```
log.error("Order failed: {}", e.getMessage());
```

This loses the stack trace.

Better:

```
log.error("Order processing failed: orderId={}", orderId, e);
```

Passing the exception as the final argument lets the logger print the stack trace.

## 10. Runtime Troubleshooting Mindset

When diagnosing an issue from logs, ask:

1. What request or job triggered the problem?
2. Is there a correlation ID?
3. What was the first warning or error?
4. Did the input look valid?
5. Did an external dependency fail?
6. Did the database call fail?
7. Is this one request, one user, or many users?
8. Did the problem start after a deployment or configuration change?

Logs are most useful when they show cause and context, not just the final failure.

## Example: Adding Useful Logs

Original service method:

```
public Order createOrder(CreateOrderRequest request) {
    validate(request);

    Customer customer = customerRepository.findById(request.customerId())
        .orElseThrow(() -> new CustomerNotFoundException(request.customerId()));

    Order order = new Order(customer, request.items());

    return orderRepository.save(order);
}
```

Improved logged version:

```
public Order createOrder(CreateOrderRequest request) {
    log.info("Create order request received: customerId={}, itemCount={}",
        request.customerId(),
        request.items().size());

    try {
        validate(request);

        Customer customer = customerRepository.findById(request.customerId())
            .orElseThrow(() -> new CustomerNotFoundException(request.customerId()));

        Order order = new Order(customer, request.items());
        Order savedOrder = orderRepository.save(order);

        log.info("Order created successfully: orderId={}, customerId={}",
            savedOrder.getId(),
            request.customerId());

        return savedOrder;
    } catch (Exception e) {
        log.error("Order creation failed: customerId={}", request.customerId(), e);
    }
}
```

```
        throw e;
    }
}
```

Key idea: log the start, important business result, and failure context.

## Practice Exercises

### Exercise 1: Add Basic Logging

Create a simple Java service class such as `OrderService` .

Practice logging:

```
log.info("Creating order for customerId={}", customerId);
log.warn("Order has no discount applied: orderId={}", orderId);
log.error("Order creation failed: customerId={}", customerId, exception);
```

Goal: understand when to use `INFO` , `WARN` , and `ERROR` .

### Exercise 2: Replace `System.out.println()`

Take an existing Java class and replace all `System.out.println()` statements with proper SLF4J logging.

Bad:

```
System.out.println("User created: " + userId);
```

Better:

```
log.info("User created: userId={}", userId);
```

Goal: learn production-style logging.

### Exercise 3: Configure Logback

Create or edit `logback-spring.xml` .

Practice:

- Console logging.
- File logging.
- Different log patterns.
- Different levels for packages.

Example:

```
<logger name="com.example.demo" level="DEBUG"/>
<root level="INFO">
  <appender-ref ref="CONSOLE"/>
</root>
```

Goal: understand how logging behavior is configured outside Java code.

#### Exercise 4: Practice Structured Logging

Write logs using key-value style fields.

Example:

```
log.info("Payment processed: orderId={}, amount={}, status={}",
        orderId,
        amount,
        status);
```

Then search the logs for:

```
status=FAILED
orderId=123
```

Goal: make logs searchable and useful.

#### Exercise 5: Add Correlation IDs

Add a request correlation ID to a Spring Boot app.

Practice:

1. Generate a UUID per request.
2. Store it in MDC.
3. Include it in every log line.
4. Clear MDC after the request.

Expected log style:

```
correlationId=9f31a request=POST /orders message="Order created"
```

Goal: trace one request across multiple logs.

#### Exercise 6: Log Exceptions Correctly

Create a method that intentionally throws an exception.

Practice bad vs. good exception logging.

Bad:

```
log.error("Something failed: {}", e.getMessage());
```

Good:

```
log.error("Order processing failed: orderId={}", orderId, e);
```

Goal: preserve stack traces for diagnostics.

## Exercise 7: Sensitive Data Logging Audit

Given a login or payment flow, review logs and remove sensitive data.

Do not log:

- passwords
- tokens
- full credit card numbers
- private keys
- session cookies

Practice changing this:

```
log.info("Login request: username={}, password={}", username, password);
```

To this:

```
log.info("Login attempt received: username={}", username);
```

Goal: build secure logging habits.

## Exercise 8: Troubleshoot From Logs

Create a small flow:

```
create order -> validate customer -> charge payment -> save order
```

Intentionally fail one step.

Then use only logs to answer:

- Where did it fail?
- What input caused it?
- Was it validation, payment, database, or code?
- What correlation ID identifies the request?

Goal: practice real diagnostic thinking.

## Exercise 9: Change Log Levels by Environment

Configure:

- `DEBUG` logging for local development.
- `INFO` logging for production.
- Package-specific logging levels.

Example:

```
logging.level.com.example.demo=DEBUG  
logging.level.org.springframework=INFO
```

Goal: understand environment-specific diagnostics.



## Exercise 10: Mini Project

Build a tiny Spring Boot API:

```
POST /orders
GET /orders/{id}
POST /payments
```

Add:

- SLF4J logging.
- Structured key-value logs.
- Correlation ID.
- Exception logging.
- Safe logging rules.
- Logback configuration.

Goal: combine the full Module 20 skill set.

## Lab: Structured Logging and Diagnostics

### Goal

Build a small Spring Boot service and use logs to diagnose failures.

### Scenario

You are building an order service. A customer submits an order, the system validates it, processes payment, and saves the order. Your job is to add useful logs so production issues can be diagnosed quickly.

### Part 1: Create the Service

Create a Spring Boot app with one endpoint:

```
POST /orders
```

Sample request:

```
{
  "customerId": "C1001",
  "itemId": "I2001",
  "quantity": 2,
  "paymentToken": "tok_test_123"
}
```

Sample response:

```
{
  "orderId": "09001",
  "status": "CREATED"
}
```

## Part 2: Add Logging

Use SLF4J.

Log these events:

```
Order request received
Order validation started
Order validation completed
Payment processing started
Payment processing completed
Order saved
Order creation failed
```

Example:

```
log.info("Order request received: customerId={}, itemId={}, quantity={}",
        request.customerId(),
        request.itemId(),
        request.quantity());
```

Do not log `paymentToken` .

## Part 3: Add Correlation ID

For each request:

1. Generate a UUID.
2. Store it in MDC.
3. Add it to every log line.
4. Clear MDC when the request ends.

Expected log style:

```
correlationId=abc-123 level=INFO message="Order request received"
```

## Part 4: Add Failure Cases

Create three intentional failures:

1. If `quantity <= 0` , return validation error.
2. If `paymentToken` equals `"fail-payment"` , throw payment failure.
3. If `customerId` equals `"C9999"` , throw customer not found.

Log each failure clearly.

Example:

```
log.warn("Order validation failed: customerId={}, reason={}",
        request.customerId(),
        "quantity must be greater than zero");
```

## Part 5: Diagnose From Logs

Send this invalid quantity request:

```
{
  "customerId": "C1001",
  "itemId": "I2001",
  "quantity": 0,
  "paymentToken": "tok_test_123"
}
```

Send this payment failure request:

```
{
  "customerId": "C1001",
  "itemId": "I2001",
  "quantity": 2,
  "paymentToken": "fail-payment"
}
```

Send this customer not found request:

```
{
  "customerId": "C9999",
  "itemId": "I2001",
  "quantity": 2,
  "paymentToken": "tok_test_123"
}
```

For each case, answer:

```
What failed?
Where did it fail?
What was the correlationId?
Was sensitive data logged?
What log level was used?
```

## Part 6: Acceptance Criteria

Your lab is complete when:

- Logs use `INFO`, `WARN`, and `ERROR` correctly.
- Logs include useful fields like `customerId`, `itemId`, `quantity`, `orderId`, and `status`.
- Logs never include `paymentToken`.
- Each request has a correlation ID.
- Exceptions include stack traces.
- You can diagnose all three failures using logs only.

## Stretch Task

Configure separate logging levels:

```
logging.level.com.example=DEBUG
logging.level.org.springframework=INFO
```

Then add one `DEBUG` log inside the service:

```
log.debug("Validated order request object: {}", request);
```

Make sure the debug log still does not expose sensitive data.

## Quick Check

Answer these before moving to Module 21:

1. What is the difference between `INFO` , `WARN` , and `ERROR` ?
2. Why is `log.info("id={}", id)` better than string concatenation?
3. What is a correlation ID?
4. Why should we avoid logging passwords or tokens?
5. How do you correctly log an exception with its stack trace?